

Organización de datos: TP2

Integrantes:

Mia Casanovas Di Sera, Mario Esteban Arriaga, Augusto Matias Sulsente y Gabriel Seneca

Enunciado 1

Se quiere registrar todas las notas de un alumno de una facultad. Para esto se arma un registro con Código de la Universidad, Código de Facultad, Código de la Asignatura, Legajo del Alumno, Edad al momento de rendir, Código del Docente a cargo, Nota y Observaciones.

Enunciado 2

Se quiere registrar todas las compras de un local nacional de electrodomésticos. Para esto se realiza un registro de la siguiente manera: Fecha, Hora, Código de Provincia, Código de Ciudad, Código del Local, Legajo del Vendedor, Factura, Código del Producto, Cantidad, Valor Unitario, Total Renglón y Observaciones.

Enunciado 3

Se tiene un archivo maestro y de novedades (ambos ordenados por ID) con la siguiente estructura: ID E2, Descripción C25, valor F4.

1*) Para los enunciados 1 y 2 realizar las definiciones conceptuales y lógicas, tanto para estructuras de formato fijo como variable.

4*) Explique las diferencias en los procedimientos realizados en los ejercicios 2 y 3 según tipo de estructura (fija / variable).

5*) Apareo: Del enunciado 3 realizar la intersección.

6*) Apareo: Del enunciado 3 realizar la diferencia.

7*) Apareo: Del enunciado 3 realizar la unión.

8*) Del enunciado 3, qué cambios realizaría en la estructura del archivo Novedades para que acepte Altas, Bajas y Actualizaciones. Justificar.

10*) Corte de Control: Del enunciado 1 realizar el informe de promedio de notas, agrupado por alumnos y facultad.

11*) Busque al menos otros tres métodos de ordenamientos para archivos. Descríbalos y arme el pseudocódigo de los mismos. Confeccione un cuadro comparativo.

1. Longitud fija:

Código de la Universidad d C10

Código de Facultad d C10

Código de la Asignatura d C10

Legajo del Alumno i C15

Edad al momento de rendir ? E2

Código del Docente a cargo ? C20

Nota E2

Observaciones ? C50

Longitud variable:

Código de la Universidad d C20

Código de Facultad d C20

Código de la Asignatura d C20

Legajo del Alumno i C20

Edad al momento de rendir ? E2

Código del Docente a cargo ? C20

Nota + E2

Observaciones. * T

1-2 Fijo:

Fecha C10

Hora C5

Codprov E1 i

CodCiudad E2 i

CodigoLocal E2 i

Legajo E2

Factura E2

CodProd C15

Cant E2

ValorUni F2

TotRenglon F4

Observaciones C100

variable:

Fecha CV

Hora C10

Codprov E1 i

CodCiudad E2 i

CodigoLocal E2 i

Legajo E2

Factura E2

CodProd C15 +

Cant E2

ValorUni F2 +

TotRenglon F4

Observaciones T

4 En los registros de longitud fija, es mucho más simple realizar cualquiera de los cuatro procedimientos mencionados, ya que el acceso a los registros para extraer información es mucho más simple, permitiendo acceder a cualquiera sin importar si es el primero o último. Su principal desventaja es que puede desperdiciar espacio.

En cambio, la mayor virtud de los registros de longitud variable es que solo se utiliza el espacio necesario, y no hay desperdicio. Aun así, se requiere una buena definición del registro para que se pueda acceder de forma efectiva.

Sobre los procedimientos realizados en los ejercicios 2 y 3, la mayor diferencia va a estar en la complejidad de la solución, principalmente a la hora de acceder. Los archivos de longitud fija permiten el acceso directo a por ejemplo, suponiendo que el archivo del enunciado 2 está ordenado por legajo de vendedor, el registro necesario para calcular el promedio de ventas de un vendedor dado. Si se quiere hacer lo mismo en un archivo de longitud variable, se debería recorrer el archivo desde el comienzo, lo cual es menos eficiente.

5.

```
Función intersección(){
    archivo_1 = "maestro.dat";
    archivo_2 = "novedades.dat";
    archivo_3 = "final.dat";
    lector i1,i2; escritor e3;
    leer(i1,archivo_1); leer(i2,archivo _2);
    mientras que(no es el final de archivo_1 y no es el final de archivo_2){
        si (son iguales i1.ID y i2.ID){
            e3 = i2;
            escribir(e3,archivo_3);
            leer(i1,archivo_1);
            leer(i2,archivo _2);
        }
        si no{
            si(i1.ID es mayor a i2.ID)
                leer(i2,archivo_2);
            si no
                leer(i1,archivo_1);
        }
    }
    cerrar(archivo_1);
    cerrar(archivo_2);
    cerrar(archivo_3);
}
```

6.

```
Función diferencia(){
    archivo_1 = "maestro.dat";
    archivo_2 = "novedades.dat";
    archivo_3 = "final.dat";
    lector l1,l2; escritor e3;
    leer(i1,archivo_1); leer(i2,archivo _2);
    mientras que(no es el final de archivo_1 y no es el final de archivo_2){
        si (son iguales i1.ID y i2.ID){
            leer(i1,archivo_1);
            leer(i2,archivo _2);
        }
        si no{
            si(i1.ID es mayor a i2.ID){
                mientras que(i1.ID es mayor a i2.ID){
                    e3 = i2;
                    escribir(e3,archivo_3);
                    leer(i2,archivo_2);
                }
            }
            si no{
                mientras que(i2.ID es mayor a i1.ID){
                    e3 = i1;
                    escribir(e3,archivo_3);
                    leer(i1,archivo_1);
                }
            }
        }
    }
    si(es fin de archivo archivo_1 y no es fin de archivo_2){
        mientras que(no sea fin de archivo_2){
            e3 = i2;
            escribir(e3,archivo_3);
            leer(i2,archivo_2);
        }
    }
    si(es fin de archivo archivo_2 y no es fin de archivo_1){
        mientras que(no sea fin de archivo_1){
            e3 = i1;
            escribir(e3,archivo_3);
            leer(i1,archivo_1);
        }
    }
    cerrar(archivo_1);
    cerrar(archivo_2);
    cerrar(archivo_3);
}
```

7-

```
Funcion union(){
    archivo_1 = "maestro.dat";
    archivo_2 = "novedades.dat";
    archivo_3 = "final.dat";
    lector l1,l2; escritor e3;
    leer(l1,archivo_1); leer(l2,archivo_2);
    mientras que(no es el final de archivo_1 y no es el final de archivo_2){
        si (son iguales l1.ID y l2.ID){
            e3 = l2;
            escribir(e3,archivo_3);
            leer(l1,archivo_1);
            leer(l2,archivo_2);
        }
        si no{
            si(l1.ID es mayor a l2.ID){
                e3 = l2;
                escribir(e3,archivo_3);
                leer(l2,archivo_2);
            }
            si no{
                e3 = l1;
                escribir(e3,archivo_3);
                leer(l1,archivo_1);
            }
        }
    }
}
si(es fin de archivo_1 y no es fin de archivo_2){
    mientras que(no sea fin de archivo_2){
        e3 = l2;
        escribir(e3,archivo_3);
        leer(l2,archivo_2);
    }
}
si(es fin de archivo_2 y no es fin de archivo_1){
    mientras que(no sea fin de archivo_1){
        e3 = l1;
        escribir(e3,archivo_3);
        leer(l1,archivo_1);
    }
}
cerrar(archivo_1);
cerrar(archivo_2);
cerrar(archivo_3);
}
```

8. Para que el enunciado 3 admitiese altas, bajas y actualizaciones podrían agregarse criterios, como por ejemplo:

- Altas: Registros del archivo de novedades los cuales tengan un ID que no corresponda con ningún registro del archivo maestro.
- Bajas: Registros del archivo de novedades en los cuales tenga una descripción vacía y un valor improbable como NULL, 0 u otro lo que indicaría que debe eliminarse (no copiarse) del archivo maestro al archivo final.
- Actualizaciones: Registros del archivo novedades con un valor distinto al del archivo maestro en el campo Valor.

10. Asumo que es el promedio de notas de los alumnos por facultad.

(Código de la Universidad C10

Código de Facultad C10

Código de la Asignatura C10

Legajo del Alumno d) i C15

Edad al momento de rendir ? E2

Código del Docente a cargo ? C20

Nota E2

Observaciones. ? C50

Asumo que el archivo está ordenado por código de universidad, código de facultad, código de asignatura y legajo del alumno. Asumo también que se carga solo una nota por asignatura;

procedure Informe ()

{

 archivo1 = archMaestro;

 leer(registro, archMaestro);

 controlCodUni = registro.CodUniversidad;

 mientras que (registro.CodUniversidad igual a controlCodUni)

 {

```

        controlCodFacu=registro.CodFacultad;
        acumNotas=0;
        contParcialesTomados=0;
        escribir("Promedio de notas de los alumnos de la facultad :",controlCodFacu);
        mientras que(registro.CodFacultad igual a controlCodFacu)
        {
            acumNotas+=registro.nota;
            contParcialesTomados++;
            leer(registro,arch);
        }
        fwrite(acumNotas/contParcialesTomados);
    }
}

```

11. Los metodos de busqueda que seleccionamos son:

- Heap Sort
- CombSort
- SelectionSort

Heap Sort

Algoritmo HeapSort(A[1..n])
 {

 ConstruirHeapMaximo(A)

 Para i desde n hasta 2

 Intercambiar A[1] con A[i] // mover el max a lo ultimo

 tamano_heap-1

 amontonar(A, 1)

 }

Procedimiento ConstruirHeapMaximo(A)

{

 tamano_heap = n

 Para i desde $\lfloor n/2 \rfloor$ hasta 1 hacer

 amontonar(A, i)

}

Procedimiento amontonar(A, i)

```
{  
    izquierda = 2 * i  
    derecha = 2 * i + 1  
    mayor = i  
  
    Si izquierda <= tamano_heap Y A[izquierda] > A[mayor] Entonces  
        mayor = izquierda  
  
    Si derecha <= tamano_heap Y A[derecha] > A[mayor] Entonces  
        mayor = derecha  
  
    Si mayor != i  
        Intercambiar A[i] con A[mayor]  
        amontonar(A, mayor)  
}
```

CombSort

Mientras que(salto > 1) o (intercambio = Verdadero)

salto = (truncar) salto / 1.3

Si salto < 1 entonces

salto = 1

intercambio = Falso

Para i desde 0 hasta N - salto - 1

Si A[i] > A[i + salto] entonces

intercambiovalor(A[i + salto], A[i])

intercambio = Verdadero

FinAlgoritmo

SelectionSort

Algoritmo SelectionSort

Entrada: Un arreglo A con N elementos

Salida: El arreglo A ordenado de menor a mayor

```
desde i=0 hasta N-2
{
    min=i
    desde j=i+1 hasta N-1
    {
        Si A[j]< A[min]
            min=j
    }
    Si min!=i
        Intercambio(A[i],A[j])
}
Fin
```

	Heap sort	Comb Sort	Selection Sort
caracteristicas	Es eficiente para muchos datos.	Es una optimización del Bubble Sort, y es muy facil de implementar	Es simple, pero no se recomienda para grandes volúmenes de datos.
complejidad:	$O(n \log n)$	$O(n^2)$ en el peor caso, pero en promedio se acerca a $O(n \log n)$	$O(n^2)$
funcionamiento	Se transforma un conjunto de datos en un heap máximo, del cual se extrae el elemento máximo hasta que quede ordenado	Compara elementos separados por un gap que se va reduciendo hasta ser 1, momento donde se aplica Bubble Sort.	Busca el minimo o máximo en el arreglo y lo coloca en el principio y luego sigue con el resto de datos.